

問：

我應該如何有效率的判定是合法的 opcode ？

我想到的有 逐字元比對, hash, trie

還是有其他更好的資料結構？

這是一個非常核心的編譯器設計問題！你能想到 Hash 和 Trie，代表你的資料結構觀念非常紮實。

在決定用哪種資料結構之前，我們需要先釐清 **SIC 組譯器 (OPTAB)** 的情境特性：

1. **資料量極小且固定**：標準 SIC 只有 26 個指令，就算加上 SIC/XE 也大約只有 59 個。
2. **查表極度頻繁**：Pass 1 和 Pass 2 每一行程式碼都要查一次。
3. **靜態資料 (Static)**：指令表在程式執行期間**不會被修改**（沒有新增/刪除的需求）。

基於以上特性，以下是對你提出的想法以及其他常見解法的實戰分析：

1. 陣列二分搜尋 (Sorted Array + Binary Search) 🏆 最推薦的實務首選

雖然你沒提到，但這通常是寫 C 語言 SIC 組譯器時 **CP 值最高**的作法。

- **為什麼好？** 因為 C 語言標準庫 (stdlib.h) 內建了 bsearch 函數！你完全不需要自己刻資料結構。再者，因為資料連續存放在陣列中，**Cache Locality (快取命中率) 極佳**。
- **效能：** $O(\log N)$ 。對於 60 個指令，最多只需比對 6 次字串，效能幾乎等同 Hash。
- **實作示範：**

C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
// 定義 Opcode 結構
```

```
typedef struct {
    char mnemonic[8];
    unsigned char opcode;

    int format; // 指令格式 (SIC 標準通常是 3, XE 會有 1, 2, 3/4)
} OpcodeEntry;
```

```
// ⚠ 注意：這個陣列必須「嚴格依照字母順序」排列，bsearch 才會動！
```

```
OpcodeEntry OPTAB[] = {
    {"ADD", 0x18, 3},
    {"AND", 0x40, 3},
```

```

        {"COMP", 0x28, 3},
        {"DIV", 0x24, 3},
        {"J", 0x3C, 3},
        {"JEQ", 0x30, 3},
        {"LDA", 0x00, 3},
        {"LDX", 0x04, 3},
        {"RSUB", 0x4C, 3},
        {"STA", 0x0C, 3},
        {"STX", 0x10, 3}
    // ... 補齊所有 SIC 指令
};

// 給 bsearch 用的比較函數
int compare_opcode(const void *a, const void *b) {
    const char *target = (const char *)a;
    const OpcodeEntry *entry = (const OpcodeEntry *)b;
    return strcmp(target, entry->mnemonic);
}

// 查詢函數
OpcodeEntry* find_opcode(const char *mnemonic) {
    int num_opcodes = sizeof(OPTAB) / sizeof(OPTAB[0]);
    return (OpcodeEntry*) bsearch(mnemonic, OPTAB, num_opcodes,
    sizeof(OpcodeEntry), compare_opcode);
}

```

2. 雜湊表 (Hash Table) 🏆 效能王，但稍嫌麻煩

- **分析：**尋找時間 $O(1)$ ，理論上最快。
- **缺點：**C 語言沒有內建 Hash Map。為了區區 60 個指令，你需要自己寫 Hash Function（例如 djb2）、配置陣列、處理 Collision（碰撞，例如用 Linked List 串接）。
- **結論：**除非你想順便練習 C 語言實作 Hash Table，或者你的老師特別規定使用，否則程式碼的複雜度會稍微偏高（殺雞用牛刀）。

💡 **進階冷知識：**在業界如果遇到這種「固定關鍵字」的查詢，通常會使用 **Perfect Hashing (完美雜湊)** 工具（如 *gperf*），它可以自動產出沒有碰撞的 C 語言 Hash 查詢程式碼，達到真正的 $O(1)$ 。

3. 字典樹 (Trie) ❌ 不建議在此使用

- **分析：**尋找時間為字串長度 $O(L)$ ，SIC 指令最長大概 6 個字元，看似很快。

- 缺點：
 1. 記憶體破碎/浪費：Trie 每個節點都要包含 26 個字母的指標，這會消耗大量記憶體。
 2. 指標跳躍 (Pointer Chasing)：每次走訪節點都要跟隨指標去抓記憶體位置，對於現代 CPU 快取非常不友善，實際跑起來可能比陣列二分搜尋還要慢。
 - 結論：Trie 適合用在「前綴搜尋 (Prefix Search)」或「自動完成 (Auto-complete)」，對於「完全比對」且數量少的情況並不划算。
-

4. 逐字元比對 (Linear Search / strcmp + for 迴圈) 🏆 堪用但暴力

- 分析：尋找時間 $O(N)$ 。如果 Opcode 有 60 個，最差要比對 60 次。
 - 結論：因為 N 很小，以現代電腦的運算速度，你其實「感覺不出延遲」。但從演算法的角度來看比較不優雅。如果你追求快速把作業生出來，這也是個可接受的保底方案。
-

總結建議

我會強烈建議你使用「寫死的排序陣列 + bsearch」(選項 1)。

它具備 Hash 的速度感，又不需要處理指標與碰撞問題，程式碼乾淨且非常符合 C 語言的系統程式風格。

如果你決定採用陣列與 bsearch，下一步我們就會在 Pass 1 中利用這個查詢函數來計算 LOCCTR (Location Counter) 了。你需要我稍微說明一下 LOCCTR 在遇到指令和虛擬指令 (RESW, BYTE 等) 時的累加邏輯嗎？